



INF01194 | Actividad 2

Optimización paralela del problema de la mochila

Fecha de entrega: domingo 10 de mayo de 2026

Porcentaje de la actividad: 20 % de la asignatura

Lenguaje sugerido: C++ con OpenMP (En caso de utilizar otro lenguaje, este debe ser aprobado por el profesor.)

Modalidad de evaluación: código, informe técnico y presentación con defensa.

1. Objetivo de la actividad

Implementar, paralelizar y analizar un algoritmo genético para resolver una versión extendida del problema de la mochila. El foco principal de la actividad no es solamente obtener una solución de alto valor, sino demostrar comprensión sobre qué partes del algoritmo pueden paralelizarse, qué limitaciones aparecen al usar múltiples hilos y cómo evaluar correctamente el impacto del paralelismo en términos de tiempo, *speed-up*, eficiencia y calidad de solución.

La actividad considera tres componentes evaluados:

- **Código:** implementación secuencial, implementación paralela e implementación mediante modelo de islas.
- **Informe técnico:** formulación, metodología experimental, resultados, análisis y conclusiones.
- **Presentación y defensa:** explicación oral del diseño, resultados y decisiones técnicas.

2. Contexto del problema

El problema clásico de la mochila consiste en seleccionar un subconjunto de ítems, donde cada ítem posee un valor y un peso, de manera que el valor total sea máximo sin exceder la capacidad de la mochila.

Sea un conjunto finito N de n ítems. Cada ítem i tiene un valor v_i y un peso w_i . La mochila posee una capacidad máxima de peso W . Se define una variable binaria x_i para cada ítem:

$$x_i = \begin{cases} 1, & \text{si el ítem } i \text{ es seleccionado,} \\ 0, & \text{si el ítem } i \text{ no es seleccionado.} \end{cases}$$

La formulación clásica del problema es:

$$\begin{aligned} \text{Maximizar} \quad & Z = \sum_{i=1}^n v_i x_i \\ \text{sujeto a} \quad & \sum_{i=1}^n w_i x_i \leq W, \\ & x_i \in \{0, 1\}, \quad i = 1, 2, \dots, n. \end{aligned}$$

3. Versión extendida de la mochila

Para esta actividad no se trabajará únicamente con la mochila 0/1 clásica. Cada grupo deberá implementar una versión extendida, donde cada ítem posee:

- **ID:** identificador único del ítem.
- **Valor:** beneficio asociado al ítem.
- **Peso:** peso ocupado por el ítem.
- **Volumen:** espacio ocupado por el ítem.
- **Categoría:** grupo al que pertenece el ítem.

Además, la mochila tendrá restricciones adicionales:

1. **Restricción de peso:** la suma de pesos no debe superar W .
2. **Restricción de volumen:** la suma de volúmenes no debe superar V .
3. **Restricciones por categoría:** algunas categorías pueden tener un mínimo o máximo de ítems seleccionados.
4. **Incompatibilidades:** ciertos pares de ítems no pueden seleccionarse simultáneamente.
5. **Dependencias:** si se selecciona un ítem, puede ser obligatorio seleccionar otro ítem asociado.

Por lo tanto, la solución ya no será válida solamente por cumplir el peso máximo. También deberá respetar volumen, categorías, incompatibilidades y dependencias.

4. Representación de una solución

Cada individuo del algoritmo genético será representado como un cromosoma binario de largo n :

$$X = (x_1, x_2, x_3, \dots, x_n)$$

Donde cada posición indica si el ítem correspondiente está incluido o no en la mochila.

Ejemplo:

$X = 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0$

En este caso, se seleccionan los ítems 0, 2, 3 y 6.

5. Función de aptitud

La función de aptitud debe considerar el valor total de la solución y penalizar las restricciones incumplidas. Una forma recomendada es:

$$\text{fitness}(X) = \text{valor_total}(X) - \text{penalización_total}(X)$$

Una penalización posible es:

$$\begin{aligned} \text{fitness}(X) = & \sum_{i=1}^n v_i x_i - \alpha \cdot \text{exceso_peso}(X) - \beta \cdot \text{exceso_volumen}(X) \\ & - \gamma \cdot \text{violaciones_categoría}(X) - \delta \cdot \text{incompatibilidades}(X) - \varepsilon \cdot \text{dependencias_incumplidas}(X) \end{aligned}$$

Los valores de α , β , γ , δ y ε deberán ser definidos y justificados por cada grupo. Una penalización demasiado baja puede permitir soluciones inválidas; una penalización excesivamente alta puede impedir explorar soluciones cercanas a la factibilidad.

5.1 Restricciones duras y restricciones blandas

Para esta actividad se deberá distinguir entre restricciones duras y restricciones blandas.

- **Restricciones duras:** son aquellas que la solución final reportada debe cumplir obligatoriamente. En esta actividad, se consideran restricciones duras el peso máximo W , el volumen máximo V , las incompatibilidades entre ítems y las dependencias obligatorias entre ítems.
- **Restricciones blandas:** son condiciones que pueden ser incumplidas temporalmente durante la evolución del algoritmo genético, pero que deben recibir una penalización en la función de aptitud. Las restricciones por categoría podrán ser tratadas como blandas cuando el grupo justifique que desea favorecer ciertas composiciones sin descartar inmediatamente soluciones cercanas.

Durante la ejecución del algoritmo genético se permitirá que existan individuos que violen una o más restricciones. Estos individuos no deberán ser eliminados automáticamente, sino evaluados mediante penalizaciones. Esto permite que el algoritmo explore soluciones cercanas a regiones factibles y que, mediante cruzamiento o mutación, puedan transformarse en soluciones válidas de buena calidad.

Sin embargo, la solución final informada por cada grupo deberá ser factible respecto de las restricciones duras. Por lo tanto, si el individuo con mayor *fitness* viola alguna restricción dura, el grupo deberá reportar como resultado final la mejor solución factible encontrada durante la ejecución.

En el informe se deberá indicar explícitamente:

- qué restricciones fueron tratadas como duras;
- qué restricciones fueron tratadas como blandas;
- cómo se calculó la penalización asociada a cada violación;
- cómo se verificó que la solución final reportada es factible.

6. Variantes obligatorias

Cada grupo deberá implementar y comparar las siguientes variantes.

6.1 Variante 1: Algoritmo genético secuencial

Debe incluir, como mínimo:

- Generación de población inicial.
- Evaluación de aptitud.
- Selección por torneo.
- Cruzamiento.
- Mutación.
- Elitismo.
- Criterio de término por número de generaciones o convergencia.

Esta variante será la línea base para comparar el rendimiento de las versiones paralelas.

6.2 Variante 2: Algoritmo genético paralelo con OpenMP

Se debe implementar una versión paralela usando OpenMP. Se deberán paralelizar, analizar y justificar al menos las siguientes partes:

- Evaluación de la función de aptitud para la población.
- Aplicación de operadores de cruzamiento y mutación.
- Selección por torneo, cuando corresponda.

No basta con agregar directivas `#pragma omp parallel for`. El informe deberá explicar por qué cada zona es paralelizable, qué datos son compartidos, qué datos son privados y cómo se evitaron condiciones de carrera.

6.3 Variante 3: Modelo de islas

Se debe implementar un modelo de islas, donde múltiples subpoblaciones evolucionan de forma independiente y realizan migraciones ocasionales de individuos.

Cada grupo deberá definir y justificar:

- Número de islas.
- Tamaño de población por isla.
- Frecuencia de migración.
- Cantidad de individuos migrantes.
- Criterio para seleccionar migrantes.
- Topología de migración: anillo, aleatoria, maestro-esclavo u otra.

Ejemplo de migración:

Cada 25 generaciones, cada isla envía sus 2 mejores individuos hacia la isla siguiente usando una topología de anillo.

7. Diseño experimental mínimo

Cada grupo deberá ejecutar experimentos suficientes para comparar rendimiento y calidad de solución. Como mínimo, se solicita:

- **Tres tamaños de instancia:** pequeña, mediana y grande.
- **Cantidad de hilos:** 1, 2, 4 y 8 hilos. Si el equipo posee más núcleos disponibles, se recomienda agregar 16 hilos.

- **Repeticiones:** al menos 15 ejecuciones por cada configuración experimental.
- **Semillas:** las ejecuciones deben usar semillas registradas para permitir reproducibilidad.

Tamaños sugeridos:

Instancia	Cantidad de ítems sugerida	Objetivo
Pequeña	100 ítems	Verificación funcional y depuración
Mediana	1.000 ítems	Comparación básica de rendimiento
Grande	10.000 ítems	Evaluación real del paralelismo

Para evitar restricciones triviales o imposibles, la capacidad máxima de peso W y la capacidad máxima de volumen V deberán calcularse como un porcentaje del peso total y del volumen total de todos los ítems disponibles.

Se recomienda utilizar:

$$W = 0.40 \times \text{suma_total_pesos} \quad V = 0.40 \times \text{suma_total_volúmenes}$$

Para instancias fáciles podrá usarse un porcentaje entre 50 % y 60 %. Para instancias medias, entre 35 % y 50 %. Para instancias difíciles, entre 20 % y 35 %.

No se aceptarán configuraciones donde la capacidad permita seleccionar casi todos los ítems, ni configuraciones donde no exista una solución factible.

8. Métricas obligatorias

El informe debe incluir, como mínimo, las siguientes métricas:

- Tiempo promedio de ejecución.
- Desviación estándar del tiempo.
- Mejor valor factible encontrado.
- Mejor fitness obtenido.
- Porcentaje de soluciones factibles.
- Speed-up.
- Eficiencia paralela.

El *speed-up* se calcula como:

$$S_p = \frac{T_1}{T_p}$$

Donde T_1 es el tiempo de la versión base con 1 hilo y T_p es el tiempo usando p hilos.

La eficiencia paralela se calcula como:

$$E_p = \frac{S_p}{p}$$

9. Formato sugerido de entrada

Cada grupo puede diseñar su propio formato de entrada, pero debe documentarlo claramente. Se recomienda trabajar con archivos separados:

```
items.csv
id,valor,peso,volumen,categoria

category_rules.csv
categoria,minimo,maximo

incompatibilities.csv
id_item_a,id_item_b

dependencies.csv
id_item,id_requerido
```

El programa deberá permitir seleccionar instancia, cantidad de hilos, variante y semilla desde la línea de comandos.

Ejemplo:

```
./mochila_ga --instance data/medium \
             --variant islands \
             --threads 8 \
             --seed 123
```

10. Entregables

Cada grupo deberá entregar:

1. Código fuente en C++ con OpenMP.
2. README con instrucciones claras de compilación y ejecución.
3. Instancias utilizadas o generador de instancias.
4. Archivo CSV con los resultados experimentales.
5. Informe en PDF.

Se recomienda una estructura de carpetas como la siguiente:

```
actividad_mochila_paralela/  
|-- src/  
| |-- main.cpp  
| |-- genetic_algorithm.cpp  
| |-- genetic_algorithm.hpp  
| |-- fitness.cpp  
| |-- fitness.hpp  
| |-- island_model.cpp  
| |-- island_model.hpp  
| |-- instance_loader.cpp  
| |-- instance_loader.hpp  
|  
|-- data/  
| |-- small/  
| |-- medium/  
| |-- large/  
|  
|-- results/  
| |-- resultados.csv  
|  
|-- report/  
| |-- informe.pdf  
|  
|  
|-- README.md
```

Compilación sugerida:

```
g++ -O2 -fopenmp src/*.cpp -o mochila_ga
```


11. Pauta de evaluación del código

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Algoritmo genético secuencial	10 %	No implementa una versión secuencial funcional o el programa no ejecuta.	Implementa una versión incompleta, con errores relevantes en selección, cruzamiento, mutación o evaluación.	Implementa una versión funcional, pero con detalles mejorables en elitismo, criterio de término o control de parámetros.	Implementa correctamente población, evaluación, selección, cruzamiento, mutación, elitismo y criterio de término.
Modelado de restricciones extendidas	12 %	Solo considera la mochila clásica por peso o ignora las restricciones solicitadas.	Considera algunas restricciones, pero de forma parcial, sin distinguir claramente restricciones duras y blandas.	Considera la mayoría de las restricciones y distingue restricciones duras y blandas, con errores menores o casos borde no tratados.	Modela correctamente peso, volumen, categorías, incompatibilidades y dependencias, distinguiendo restricciones duras, restricciones blandas y factibilidad final.
Función de aptitud y penalizaciones	12 %	No define una función de aptitud coherente o no penaliza soluciones inválidas.	Define penalizaciones básicas, pero sin justificar su relación con restricciones duras o blandas.	Define una función de aptitud funcional y penaliza violaciones, aunque con justificación incompleta de los pesos de penalización o de la factibilidad final.	Diseña una función de aptitud clara, penaliza adecuadamente, justifica los parámetros utilizados y reporta la mejor solución factible cuando corresponde.
Paralelización de la evaluación de fitness	14 %	No paraleliza la evaluación o la implementación produce resultados incorrectos.	Usa OpenMP de forma superficial, con bajo control de datos compartidos o privados.	Paraleliza correctamente la evaluación, aunque con oportunidades de mejora en eficiencia.	Paraleliza la evaluación de forma correcta, segura y justificada, evitando condiciones de carrera.
Paralelización de operadores genéticos	10 %	No paraleliza cruzamiento ni mutación.	Paraleliza parcialmente, pero con errores de sincronización o aleatoriedad.	Paraleliza los operadores principales con resultados correctos, aunque con justificación limitada.	Implementa operadores paralelos correctamente, con manejo adecuado de aleatoriedad por hilo.
Selección por torneo y control de concurrencia	8 %	No implementa selección por torneo o la selección es incorrecta.	Implementa torneo básico, pero no analiza problemas de concurrencia.	Implementa torneo funcional y parcialmente optimizado.	Implementa torneo correctamente, considerando acceso seguro a datos, semillas y paralelización cuando corresponde.
Modelo de islas	14 %	No implementa modelo de islas.	Implementa subpoblaciones, pero sin migración funcional o con errores graves.	Implementa islas y migración, aunque con parámetros poco justificados.	Implementa un modelo de islas completo, con migración, topología y parámetros justificados.
Reproducibilidad y medición	10 %	No registra semillas, tiempos ni configuraciones.	Registra algunos datos, pero no permite reproducir los experimentos.	Registra semillas, tiempos y configuraciones principales, con pequeños vacíos.	Permite reproducir los experimentos mediante semillas, parámetros de ejecución y resultados exportados.

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Orden, compilación y documentación técnica	10 %	Código desordenado, sin README o imposible de compilar.	Código compilable solo con ajustes importantes o documentación insuficiente.	Código ordenado y compilable, con README básico.	Código modular, compilable, documentado y con instrucciones claras de ejecución.
Total	100 %	Cada criterio se evalúa de 0 a 3 puntos y se pondera según el porcentaje indicado.			

12. Pauta de evaluación del informe técnico

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Introducción y contexto del problema	10 %	No presenta el problema o la introducción es irrelevante.	Presenta una descripción muy general, con poca conexión con optimización o paralelismo.	Presenta adecuadamente el problema y su relación con optimización.	Contextualiza claramente la mochila, sus aplicaciones, complejidad y relación con paralelismo.
Formulación matemática extendida	18 %	No formula matemáticamente el problema.	Formula solo la mochila clásica, sin incorporar restricciones adicionales.	Formula parcialmente la versión extendida, con algunas restricciones descritas informalmente o sin clasificarlas como duras/blandas.	Formula claramente variables, función objetivo, restricciones extendidas, restricciones duras, restricciones blandas y criterio de factibilidad final.
Descripción del algoritmo genético	15 %	No describe el algoritmo implementado.	Describe el algoritmo de forma superficial o incompleta.	Describe las etapas principales, pero con bajo detalle de parámetros.	Explica representación, población, selección, cruzamiento, mutación, elitismo, término y parámetros.
Estrategias de paralelización	20 %	No explica qué partes fueron paralelizadas.	Menciona directivas OpenMP sin justificar decisiones.	Explica las zonas paralelizadas y algunos problemas de concurrencia.	Justifica técnicamente cada zona paralelizada, datos compartidos/privados, sincronización y overhead.
Diseño experimental	17 %	No presenta diseño experimental claro.	Presenta experimentos insuficientes o sin control de hilos, semillas o repeticiones.	Presenta un diseño aceptable, pero con alguna omisión en hardware, repeticiones o instancias.	Define claramente instancias, hilos, repeticiones, semillas, hardware, variantes y procedimiento experimental.
Resultados, tablas y gráficos	20 %	No presenta resultados cuantitativos.	Presenta resultados incompletos o poco claros.	Presenta tablas y gráficos suficientes, aunque con interpretación limitada.	Presenta resultados completos, ordenados, con tablas, gráficos y métricas obligatorias.
Total	100 %	Cada criterio se evalúa de 0 a 3 puntos y se pondera según el porcentaje indicado.			

13. Pauta de evaluación de la presentación y defensa

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Claridad al explicar el problema	15 %	No explica el problema o lo presenta incorrectamente.	Explica solo la mochila clásica, sin considerar restricciones extendidas.	Explica el problema extendido, aunque con algunos vacíos sobre restricciones duras, blandas o factibilidad.	Explica claramente objetivo, variables, restricciones duras, restricciones blandas, factibilidad final y dificultad del problema.
Explicación del algoritmo genético	15 %	No logra explicar el algoritmo implementado.	Explica etapas aisladas sin conexión clara.	Explica las etapas principales con ejemplos básicos.	Explica con claridad representación, operadores, parámetros y decisiones de diseño.
Explicación del paralelismo	25 %	No identifica qué fue paralelizado.	Menciona OpenMP sin explicar su impacto.	Explica las zonas paralelizadas y algunos resultados de rendimiento.	Justifica técnicamente paralelización, sincronización, overhead, escalabilidad y límites observados.
Presentación de resultados	15 %	No presenta resultados o los resultados son inconsistentes.	Presenta resultados mínimos, sin gráficos o con interpretación débil.	Presenta resultados claros, aunque con análisis incompleto.	Presenta tablas, gráficos y conclusiones comparativas entre variantes e hilos.
Defensa técnica individual	20 %	No responde preguntas técnicas básicas.	Responde parcialmente, mostrando dependencia del código o de terceros.	Responde la mayoría de las preguntas con fundamentos aceptables.	Responde con seguridad sobre código, fórmulas, OpenMP, métricas, errores y decisiones técnicas.
Participación del grupo	10 %	La presentación recae en una sola persona o hay integrantes sin participación.	Participación muy desigual y con poco dominio individual.	Participación relativamente equilibrada, con diferencias menores.	Todos los integrantes participan y demuestran dominio del trabajo realizado.
Total	100 %	Cada criterio se evalúa de 0 a 3 puntos y se pondera según el porcentaje indicado.			